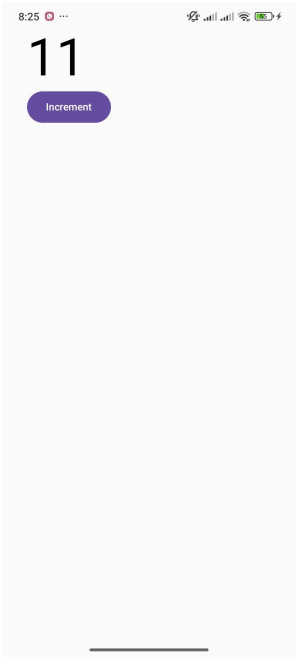
Nama: Vara Vaulina Safitri
NIM: 245150401111017

Modul 4
State Hoisting, Perubahan Konfigurasi, dan Side-effect

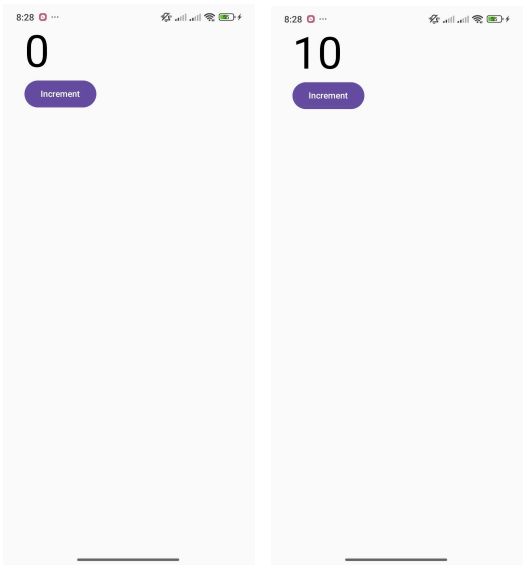
A. State Hosting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.
    a. Apa yang terjadi setiap kali tombol "Increment" diklik? Angka yang ditampilkan pada teks di atas tombol bertambah 1 angka setiap kali tombol diklik.



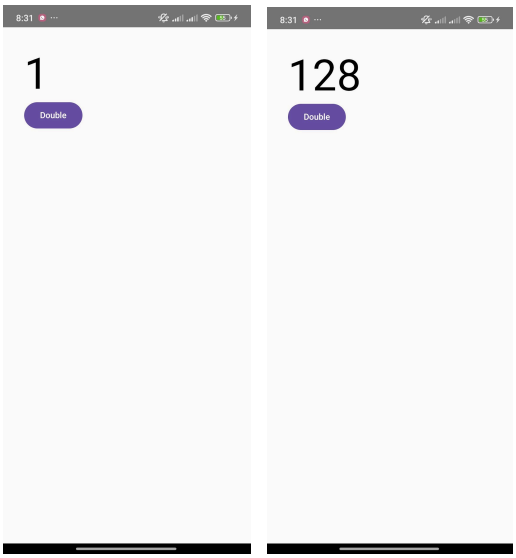
    b. Apa yang menyebabkan hal tersebut terjadi? Terjadi proses Rekomposisi (Recomposition) pada Jetpack Compose. Ketika tombol diklik, nilai variabel state number berubah (number++). Compose secara otomatis mendeteksi perubahan state ini dan menggambar ulang (re-render) elemen Text yang membaca nilai number tersebut.
    c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut? Variabel number (var number by remember { mutableStateOf(0) })
    d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut? Variabel label (val label by remember { mutableStateOf("Increment") })

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".
    a. Perubahan apa yang perlu dilakukan dan dilakukan di mana? Pada tahap ini, kita akan mengubah CounterScreen dari Stateful (mengelola data sendiri) menjadi Stateless (menerima data dari luar) melalui teknik State Hoisting
    b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.



3. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
    a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya? Ya, program berjalan dengan baik dan menampilkan perilaku visual yang sama persis
    b. Di bagian kode program manakah saat ini proses increment dilakukan? Proses increment dilakukan pada pemanggil fungsi (onCreate / parent composable) di dalam lambda callback onButtonClick = { number++ }
    c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan? Variabel state number dideklarasikan di dalam fungsi onCreate() (induk), lalu nilainya diteruskan ke CounterScreen melalui parameter number dan label
4. Jalankan program,

a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

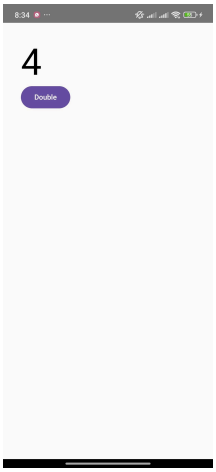


b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan? Perubahan hanya dilakukan di dalam fungsi onCreate() (pada parent composable)
c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Tidak ada. Fungsi CounterScreen sama sekali tidak diubah

5. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna? Ini membuktikan keunggulan utama dari State Hoisting, yaitu reusability (kemampuan untuk digunakan kembali) dan Separation of Concerns (pemisahan logika). Komponen CounterScreen menjadi stateless sehingga sangat fleksibel. Kita dapat mengubah logika bisnis (menambah, menggandakan, mengganti nilai awal, mengganti label) cukup dari komponen induk tanpa perlu menyentuh atau merusak kode UI di CounterScreen

B. Perubahan Konfigurasi

1. Jalankan program, dengan memastikan posisi perangkat pada posisi ***portrait***, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.
   Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

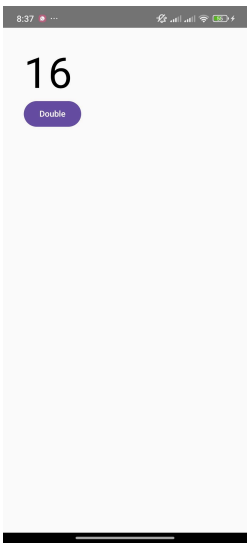




2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
    a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.
    b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

        Angka kembali ke 1 (nilai awal). Hal ini terjadi karena saat layar dirotasi, Android melakukan Configuration Change yang menghancurkan (destroy) dan membuat ulang (recreate) Activity. Konstruk remember hanya menyimpan nilai state selama proses rekomposisi, tetapi tidak dapat bertahan ketika Activity dibuat ulang akibat perubahan konfigurasi.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.





a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.
b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

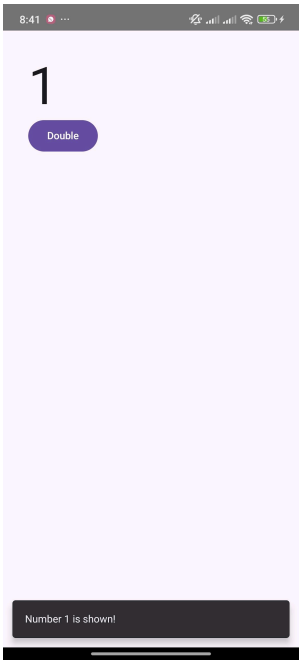
   Angka tetap bertahan pada 16 (tidak kembali ke 1). Hal ini terjadi karena rememberSaveable secara otomatis menyimpan nilai state ke dalam objek Bundle (SavedInstanceState). Ketika Activity dibuat ulang akibat rotasi layar, state dipulihkan kembali dari Bundle tersebut

4. Apa fungsi dari konstruk rememberSavable pada kode program Android yang membedakannya dari konstruk remember?
   - remember: Hanya mempertahankan state selama proses rekomposisi UI.
   - rememberSaveable: Mempertahankan state baik saat rekomposisi maupun saat perubahan konfigurasi (seperti rotasi layar atau perubahan tema) dengan menyimpan data ke dalam Bundle
5. Beri contoh kasus nyata lain di mana penggunaan konstruk rememberSavable ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!
   - Contoh Kasus: Formulir pendaftaran (input text field), checkbox persetujuan syarat & ketentuan, pencarian data (search bar), atau status tab yang sedang dipilih.
   - Manfaat: Mencegah pengguna kehilangan data yang sudah diketik/diisi secara tidak sengaja ketika memutar layar atau merubah tema aplikasi (Dark/Light mode), sehingga meningkatkan User Experience (UX)

C. Launched Effect

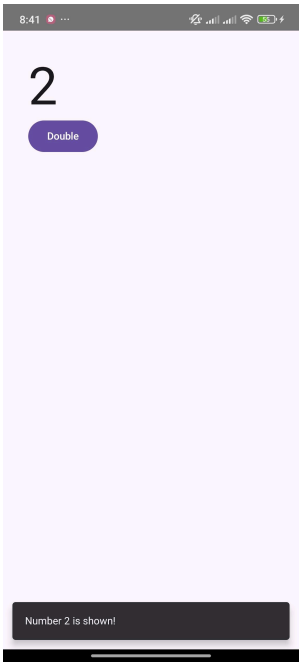
❖ Percobaan 1

1. Jalankan program,
   a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar? Layar menampilkan angka 1, tombol "Double", dan sebuah Snackbar muncul di bagian bawah layar dengan teks "Number 1 is shown!"



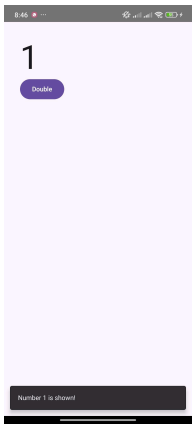
   b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna? Karena LaunchedEffect selalu dieksekusi minimal satu kali saat komponen composable pertama kali masuk ke dalam komposisi UI (Initial Composition).
   c. Apa yang memicu pesan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana? Dipicu oleh fungsi LaunchedEffect(number) pada baris: LaunchedEffect(number) { snackbarHostState.showSnackbar("Number $number is shown!") }

2. Klik button "Double" satu kali,
   a. Apa yang terjadi? Mengapa hal tersebut terjadi? Jelaskan! Angka berubah menjadi 2 dan Snackbar baru muncul dengan teks "Number 2 is shown!". Hal ini terjadi karena nilai key number berubah (dari 1 menjadi 2). LaunchedEffect mendeteksi perubahan key tersebut, membatalkan effect sebelumnya, dan menjalankan ulang blok kode di dalamnya.

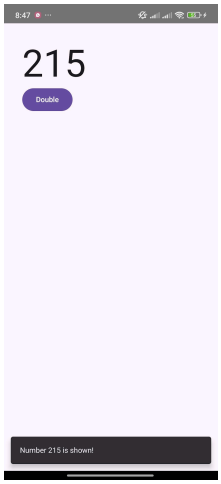


❖ Percobaan 2

1. Jalankan program,
    a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar? Layar awalnya menampilkan angka 1 dan Snackbar bertuliskan "Number 1 is shown!"



    b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil? Setelah 2 detik, angka 1 berubah secara otomatis menjadi angka acak antara 100 - 1000 (misal: 487), dan Snackbar baru muncul dengan teks "Number 487 is shown!"



c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana? Dipicu oleh LaunchedEffect(Unit) pada baris number = Repo.getData(). Setelah coroutine menunda 2 detik (delay(2000)), variabel number diisi dengan angka acak baru. Perubahan number ini secara otomatis juga memicu LaunchedEffect(number) untuk menampilkan Snackbar baru.

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,
   a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?
      - LaunchedEffect(Unit): Menggunakan constant key (Unit). Blok kode di dalamnya hanya dijalankan 1 kali saja saat komponen UI pertama kali muncul (enter composition). Digunakan untuk inisialisasi awal seperti mengambil data dari API saat layar dibuka.
      - LaunchedEffect(state): Menggunakan variabel state sebagai key. Blok kode akan dijalankan saat pertama kali muncul DAN akan dijalankan ulang setiap kali nilai variabel state tersebut mengalami perubahan.
   b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?
      - LaunchedEffect(Unit): Dijalankan saat composable pertama kali dipasang pada UI Tree.
      - LaunchedEffect(state): Dijalankan saat pertama kali dipasang pada UI Tree serta setiap kali variabel state kuncinya berubah nilai.